ACM-ICPC 个人算法模板

算法竞赛常用代码与知识点汇总

整理人：halfpoint

适用场景：ACM-ICPC、CCPC、蓝桥杯、牛客算法竞赛等

代码语言：C++

目 录

章节	标题	页码
一	字符串哈希（hash_sum）	3
二	树的基础数据结构与算法	4
2.1	树的基础概念	4
2.2	前中后序遍历	5
2.3	并查集	8
2.4	MST（最小生成树）	9
2.4.1	Prim算法	9
2.4.2	Kruskal算法	11
2.5	LCA（最近公共祖先）	13
2.5.1	树链剖分	13
2.5.2	倍增	14
三	单调队列 & 单调栈 核心笔记	15
3.1	单调队列	15

章节	标题	页码
3.2	单调栈	16
四	ST表（倍增）	17
五	字符串算法（KMP + Manacher）	19
5.1	KMP算法	19
5.2	Manacher（马拉车算法）	21
六	数论基础	22
6.1	费马小定理笔记	22
6.2	等比数列求和公式笔记	24
七	筛法（欧拉筛）	26
八	欧拉路径与欧拉回路	27

一、字符串哈希（hash_sum）

用法

维护字符串区间哈希值，快速比较子串是否相等、判断回文等。

- 预处理时间：$\mathcal{O}(n)$
- 查询时间：$\mathcal{O}(1)$

实现步骤

1. 生成随机基数 `base`（用 `mt19937_64` 生成，防卡哈希）
2. 预处理 `base` 的幂次数组 `power`，其中 `power[i] = base^i`
3. 预处理前缀哈希数组 `hash_pre`，其中 `hash_pre[i]` 表示前 `i` 个字符的哈希值
4. 查询区间 `[l, r]`（1-based）的哈希值：`hash_pre[r] - hash_pre[l-1] * power[r-l+1]`

完整模板

```cpp
#include <bits/stdc++.h>
#include <random>
#include <chrono>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

const int MAXN = 1e5 + 5;
u64 base;
u64 power[MAXN];
u64 hash_pre[MAXN];

void init_hash() {
    base = rng();
    power[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        power[i] = power[i-1] * base;
    }
}

void build_hash(const string &s) {
    int n = s.size();
    for (int i = 0; i < n; i++) {
        hash_pre[i+1] = hash_pre[i] * base + s[i];
    }
}

u64 get_hash(int l, int r) {
    return hash_pre[r] - hash_pre[l-1] * power[r - l + 1];
}

int main() {
    init_hash();
    string s;
    cin >> s;
    build_hash(s);
    int l = 2, r = 4;
    cout << get_hash(l, r) << endl;
    return 0;
}
```


二、树的基础数据结构与算法

2.1 树的基础概念

树的定义



结点间的关系

- 双亲和孩子
  
- 兄弟
  

- 祖先
  
- 子孙
  

树的层数

下图共四层


高度与深度

- 对树来说：高度 = 深度 = 层数
- 对结点来说：
  - 高度 = 从下往上
  - 深度 = 从上往下
    

度

- 结点的度 = 该结点的分支数
  - 度 = 0 时，结点为叶结点
  - 度 > 0 时，称为分支结点
    
- 树的度 = 树里最大的结点度
  

m叉树和m度树的区别

叉是最大的上限，度是最大的下限


有序树和无序树

各结点子树从左到右有序




路径

从上到下经过的结点数


森林

m(m >= 0)颗树的集合


常见性质

1. 结点数n = 边数 + 1 = 所有结点的度数和 + 1
2. m叉树(设层数为i)的每一层最多的结点数为m ^ (i - 1)
3. 具有n个结点的m叉树的最小高度
   

2.2 前中后序遍历

用法

按照不同顺序访问树的所有结点，分为前序（根-左-右）、中序（左-根-右）、后序（左-右-根），可通过递归或栈实现。

实现步骤

- 前序：中左右，递归序 = 1
- 中序：左中右，递归序 = 2
- 后序：左右中，递归序 = 3

递归写法：

```
void 递归() {
          if(head == NULL) {
                    break;
          }
          //输出head 前序
          递归(head.left)
          //输出head 中序
          递归(head.right)
          //输出head 后序

}
```

完整模板（栈写法）

```cpp
#include <iostream>
#include <vector>
#include <stack>
#include <algorithm>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->right) stk.push(node->right);
        if (node->left)  stk.push(node->left);
    }
    return result;
}

vector<int> inorderTraversal(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    while (cur || !stk.empty()) {
        while (cur) {
            stk.push(cur);
            cur = cur->left;
        }
        cur = stk.top();
        stk.pop();
        result.push_back(cur->val);
        cur = cur->right;
    }
    return result;
}

vector<int> postorderTraversal(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    TreeNode* prev = nullptr;
    while (cur || !stk.empty()) {
        while (cur) {
            stk.push(cur);
            cur = cur->left;
        }
        cur = stk.top();
        if (!cur->right || cur->right == prev) {
            result.push_back(cur->val);
            stk.pop();
            prev = cur;
            cur = nullptr;
        } else {
            cur = cur->right;
        }
```

```
    }
    return result;
}

vector<int> postorderTraversal_reverse(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->left)  stk.push(node->left);
        if (node->right) stk.push(node->right);
    }
    reverse(result.begin(), result.end());
    return result;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    cout << "前序遍历: ";
    for (int x : preorderTraversal(root)) cout << x << " ";
    cout << "\n中序遍历: ";
    for (int x : inorderTraversal(root)) cout << x << " ";
    cout << "\n后序遍历: ";
    for (int x : postorderTraversal(root)) cout << x << " ";
    return 0;
}
```


2.3 并查集

用法

用于动态维护和查询集合的合并与连通性，支持Find（查找根结点）和Union（合并集合）操作，路径压缩可优化查询效率至近似O(1)。

实现步骤

1. 存储结构：
   - 使用数组，下标表示结点，数组值表示该结点的父结点，根结点的父结点是自己
     
2. Find操作：
   - 普通写法：递归向上找根结点，不破坏原树结构
   - 路径压缩：递归时让路径上的每个结点直接指向根，优化后续查询
3. Union操作：找到两个元素的根结点，若不同则让一个根指向另一个根

完整模板

```
vector<int> P;

int Find(int x) {
    if(P[x] == x) {
        return x;
    }
    return Find(P[x]);
}

int Find(int x) {
    if(P[x] == x) {
        return x;
    }
    return P[x] = Find(P[x]);
}

void Union(int x, int y) {
    int rootx = Find(x);
    int rooty = Find(y);
    if(rootx != rooty) {
        P[rootx] = rooty;
    }
}
```


2.4 MST（最小生成树）

2.4.1 Prim算法

用法

在**稠密图**中求解最小生成树，从任意结点出发，通过维护当前连通块到未连通结点的最小边权，逐步扩展连通块。时间复杂度 $\mathcal{O}(m \log n)$（优先队列实现）。

实现步骤

1. 初始化距离数组 `dist[]` 为无穷大，选择任意起点（如结点1），将其距离设为0
2. 使用优先队列（小顶堆）存储 `(距离, 结点)`，将起点入队
3. 用 `vis[]` 数组维护已加入最小生成树的结点
4. 每次取出队首距离最小的未访问结点，标记为已访问，累加其距离到总权值
5. 遍历该结点的所有邻接点，若邻接点未访问且当前边权小于其 `dist`，更新 `dist` 并入队

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const int INF = 0x3f3f3f3f;

vector<vector<pair<int, int>>> G;
vector<int> dist;
vector<bool> vis;

i64 prim(int n, int start = 1) {
    dist.assign(n + 1, INF);
    vis.assign(n + 1, false);
    dist[start] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    pq.emplace(0, start);
    i64 total = 0;
    int cnt = 0;
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        total += d;
        cnt++;
        for (auto [v, w] : G[u]) {
            if (!vis[v] && w < dist[v]) {
                dist[v] = w;
                pq.emplace(dist[v], v);
            }
        }
    }
    return cnt == n ? total : -1;
}

int main() {
    int n, m;
    cin >> n >> m;
    G.resize(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }
    i64 ans = prim(n);
    if (ans == -1) cout << "orz" << endl;
    else cout << ans << endl;
    return 0;
}
```


2.4.2 Kruskal算法

用法

在**稀疏图**中更优，通过将所有边按权值排序，结合并查集逐步合并连通分量，每次选择连接两个不同连通分量的最小边。时间复杂度 $\mathcal{O}(m \log m)$（主要为边排序的时间）。

实现步骤

1. 定义边的结构体，包含起点、终点、权值

2. 将所有边按权值从小到大排序
3. 初始化并查集，每个结点独立成一个集合
4. 遍历排序后的边，若边的两个端点不在同一连通分量，则合并它们，并累加该边权值到总权值
5. 当合并次数达到 `n-1` 时（已形成生成树），提前退出

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct Edge {
    int u, v, w;
    Edge(int u, int v, int w) : u(u), v(v), w(w) {}
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

vector<Edge> edges;
vector<int> P;

int Find(int x) {
    if (P[x] != x) P[x] = Find(P[x]);
    return P[x];
}

i64 kruskal(int n) {
    sort(edges.begin(), edges.end());
    P.resize(n + 1);
    for (int i = 1; i <= n; i++) P[i] = i;
    i64 total = 0;
    int cnt = 0;
    for (auto& e : edges) {
        int rootu = Find(e.u);
        int rootv = Find(e.v);
        if (rootu != rootv) {
            P[rootu] = rootv;
            total += e.w;
            cnt++;
            if (cnt == n - 1) break;
        }
    }
    return cnt == n - 1 ? total : -1;
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
    }
    i64 ans = kruskal(n);
    if (ans == -1) cout << "orz" << endl;
    else cout << ans << endl;
    return 0;
}
```

2.5 LCA（最近公共祖先）

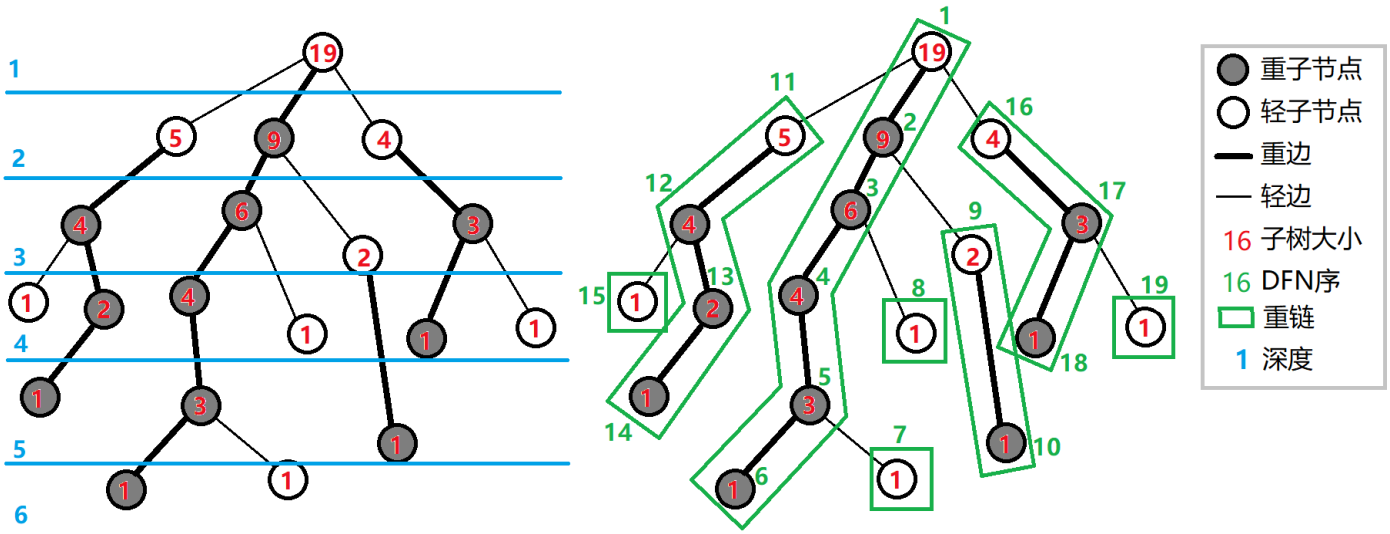
2.5.1 树链剖分

用法

预处理O(n)，查询O(logn)，通过重链剖分将树划分为若干重链，快速跳转到LCA。

实现步骤

1. 第一个DFS（dfs1）：处理父结点father、深度deepth、子树大小size、重子结点son
2. 第二个DFS（dfs2）：创建链顶top，重子结点的链顶为父结点的链顶，轻子结点的链顶为自己
3. 寻找LCA：
   - 若二者不在同一个重链上，让链顶深度大的点跳转到他链顶的父结点
   - 直到二者在同一个重链上，深度更小的点为LCA



完整模板

```
vector<vector<int>> G;
vector<int> fa, dep, siz, son, top;

void dfs1(int u, int f) {
  fa[u] = f, dep[u] = dep[f] + 1, siz[u] = 1;
  for (auto v : G[u]) {
    if (v == f) continue;
    dfs1(v, u);
    siz[u] += siz[v];
    if (siz[v] > siz[son[u]]) son[u] = v;
  }
}

void dfs2(int u, int f) {
    if (son[f] == u)
        top[u] = top[f];
    else
        top[u] = u;
    for (int v : G[u]) {
        if (v != f)
            dfs2(v, u);
    }
}

int LCA(int a, int b) {
    while(top[a] != top[b]) {
        if (dep[top[a]] > dep[top[b]]) a = fa[top[a]];
        else b = fa[top[b]];
    }
    return dep[a] <= dep[b] ? a : b;
}
```


2.5.2 倍增

用法

预处理O(nlogn)，查询O(logn)，利用二进制思想，通过倍增表快速跳转到祖先结点。

实现步骤

1. DFS：处理父亲函数fa[0][]（2^0 = 1）和深度dep
2. 倍增表初始化：第$2^{k祖先是第2}$(k-1)祖先的第2^(k-1)祖先
3. LCA函数：
   - 先将二者跳到同一深度
   - 若相同则直接返回
   - 否则从$2^{k到2}$0尝试跳转，直到找到LCA

完整模板

```
vector<vector<int>> Edge;
vector<vector<int>> fa;
vector<int> dep;
int n;

void dfs(int u, int f) {
    fa[0][u] = f;
    dep[u] = dep[f] + 1;
    for(auto &i:Edge[u]) {
        if(i != f) dfs(i, u);
    }
}

void init() {
    for(int i = 1; i <= 20; i++) {
        for(int j = 1; j <= n; j++) {
            fa[i][j] = fa[i - 1][fa[i - 1][j]];
        }
    }
}

int LCA(int a, int b) {
    if(dep[a] < dep[b]) swap(a, b);
    for(int i = 20; i >= 0; i--) {
        if(dep[fa[i][a]] >= dep[b]) a = fa[i][a];
    }
    if(a == b) return a;
    for(int i = 20; i >= 0; i--) {
        if(fa[i][a] != fa[i][b]) {
            a = fa[i][a];
            b = fa[i][b];
        }
    }
    return fa[0][a];
}
```

2.6 树状数组

用法

一切可差分的信息 都能用树状数组来维护 例如 总和 总乘

树状数组能维护

树状数组的思想

在设计之初 数组一定要从1开始
树状数组的下标代表的是该下标维护的原数组右边界
下标的lowbit是维护的数组的长度
通过这两个信息就能知道该下标的所有信息

在查询时候 通过两个前缀和就能得到需要的差分信息
反之 在修改的时候 一直加lowbit就是一直在扩展自己的范围且不会遗漏

复杂度

时间 查询 修改均为logn
初始化 一般为nlogn 如果有所有信息 通过只修改自己的上一个父数组可以优化到n

函数

树状数组一般需要四个函数

1. lowbit 函数 获取lowbit值
2. update 函数 用来更新树状数组
3. init 函数 初始化树状数组
4. query 函数 用来查找

(以下板子均为和 查询也是区间和)

完整模板

```
// 1.lowbit函数如下
i64 lowbit(int x) {
          return (x & -x);
}

// 2.update函数如下
void update(int x, int k) {
          for(int i = x; i <= n; i += lowbit(i)) {
                    c[i] += k; //c[i]为树状数组
          }
}

// 3.init函数如下 logn版本
          for(int i = 1; i <= n; i++) {
                    cin >> arr[i];
                    update(i, arr[i]);
          }

// 3.init函数如下 n版本
// 1. 预装填：把 tree 变成 a 的副本（下标从 1 开始）
for (int i = 1; i <= n; i++) {
    tree[i] = a[i];
}

// 2. 线性合并
for (int i = 1; i <= n; i++) {
    int parent = i + lowbit(i);
    if (parent <= n) {
        // 将当前节点的【最终总和】直接累加给【直接父亲】
        // 这样 parent 遍历到的时候，它就已经拿到了所有下属的值
        tree[parent] += tree[i];
    }
}

// 4.query函数如下
int query(int a) {
          int sum = 0;
          for(int i = a; i >= 1; i -= lowbit(i)) {
                    sum += c[i];
          }
          return sum;
}
```


三、单调队列 & 单调栈 核心笔记

3.1 单调队列

用法

单调队列可用以解决滑动窗口最值问题

实现步骤

单调队列的本质可以分为6步

1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候分别考虑
3. 队头是否滑出范围 是则删除到不是为止
4. 队尾是否比新来的更小或者更大 是则删除到不是为止
5. 入队新元素
6. 此时队头就是窗口的最小值

完整模板

```
deque<int> dq;
for (int i = 0; i < n; i++) {
    if (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) cout << a[dq.front()] << " ";
}
```


3.2 单调栈

用法

找左右边界、下一个大小元素、矩形 / 接雨水、子数组贡献、嵌套括号

实现步骤

单调栈的本质可以分为7步

1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候考虑
3. 新元素是不是比当前的栈头更大或者更小或其他
4. 如新元素是 则弹出头到不是为止
5. 入栈新元素
6. 此时栈头就是窗口的最小值
7. 答案数组记录下此时的位置即可

完整模板

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
int main() {
    i64 n;
    cin >> n;
    vector<i64> arr(n);
    vector<i64> ans(n, 0);
    for(auto &i:arr) {
        cin >> i;
    }
    stack<i64> S;
    for(i64 i = 0; i < n; i++) {
        while (!S.empty() && arr[S.top()] < arr[i]) {
            ans[S.top()] = i + 1;
            S.pop();
        }
        S.push(i);
    }
    for(auto &i:ans) {
        cout << i << ' ';
    }
}
```


四、ST表（倍增）

用法

用以维护**静态可重复贡献RMQ**，例如 `MAX`、 `MIN`、 `GCD`、 `LCM`、 `|`、 `&`。

时间复杂度

- 预处理时间：$\mathcal{O}(n \log n)$
- 查询时间：$\mathcal{O}(1)$

核心思想

ST表运用了**倍增**的思想：
先维护每个位置从 2^0 到 2^n（小于最后位置）长度中的RMQ。

> 例子：数组 1 2 3 4 5 6 7 8，查询 2-6 只需 2-4 和 4-6 合并。

实现步骤

1. logn预处理

```
void pre() {
         logN[2] = 1;
         for(i64 i = 3; i < n; i++) {
                  logN[i] = logN[i / 2] + 1;
         }
}
```

2. 初始化

```
void init() {
	for(i64 i = 0; i < n; i++) st[i][0] = arr[i];
	for(i64 i = 1; i <= logN[n]; i++) {
		for(i64 j = 0; j + (1 << i) <= n; j++) {
			st[j][i] = max(st[j][i - 1], st[j + (1 << (i - 1))][i - 1]);
		}
	}
}
```

3. 查询

```
i64 query(i64 l, i64 r) {
	i64 k = logN[l - r + 1];
	return max(st[l][k], st[r - (1 << k) + 1][k]);
}
```

完整板子

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
i64 n;
vector<i64> logN;
vector<i64> arr;
vector<vector<i64>> st(100, vector<i64>(100));

void pre() {
	logN[2] = 1;
	for(i64 i = 3; i < n; i++) {
		logN[i] = logN[i / 2] + 1;
	}
}

void init() {
	for(i64 i = 0; i < n; i++) st[i][0] = arr[i];
	for(i64 i = 1; i <= logN[n]; i++) {
		for(i64 j = 0; j + (1 << i) <= n; j++) {
			st[j][i] = max(st[j][i - 1], st[j + (1 << (i - 1))][i - 1]);
		}
	}
}

i64 query(i64 l, i64 r) {
	i64 k = logN[l - r + 1];
	return max(st[l][k], st[r - (1 << k) + 1][k]);
}

int main() {
	cin >> n;
	arr.resize(n + 1);
	logN.resize(n + 1);
}
```

五、字符串算法（KMP + Manacher）

5.1 KMP算法

用法

用于单模式串匹配，可在主串中快速查找模式串的所有出现位置，避免暴力匹配的回溯，时间复杂度 $\mathcal{O}(n+m)$。

实现步骤

对于一串字符串 如
匹配串: abcdabxa...
模式串: abcdabcy

可以发现 abcdab是匹配的
且abcdab的最长前后缀存在为ab
因为前面都匹配 只有从c-x不匹配
所以x前的肯定是ab
那就意味着可以从c-x开始比较 而不用从头开始遍历

完整模板

```
#include <bits/stdc++.h>
using namespace std;
int main() {
	string s1;
	string s2;
	cin >> s1 >> s2;
	int s2_len = s2.size();
	int s1_len = s1.size();
	vector<int> next(s2_len, 0);
	int i = 0, j = 1;
	while(i < s2_len && j < s2_len) {
		while (i > 0 && s2[i] != s2[j]) {
			i = next[i - 1];
		}
		if(s2[i] == s2[j]) i++;
		next[j] = i;
		j++;
	}
	i = 0;
	j = 0;
	while (j < s1_len) {
		while(i > 0 && s2[i] != s1[j]) {
			i = next[i - 1];
		}
		if(s2[i] == s1[j]) {
			i++;
		}
		if(i == s2_len) {
			cout << j - i + 2 << endl;
		}
		j++;
	}
	for(auto &i:next) {
		cout << i << ' ';
	}
}
```

5.2 Manacher（马拉车算法）

用法

用于在线性时间 $\mathcal{O}(n)$ 内求解字符串的最长回文子串长度。

实现步骤

1. 预处理字符串：插入 `#` 统一奇偶回文
2. 维护 `maxr` 、 `mid` 、 `ans`
3. 遍历中心，利用对称性扩展回文半径

完整模板

```
#include <bits/stdc++.h>
using namespace std;
string change(string &s) {
	string temp = "!#";
	for(auto &c:s) {
		temp += c;
		temp += '#';
	}
	temp += "^";
	return temp;
}

int manacher(string &s) {
	int maxr = 0, mid = 0, ans = 0;
	vector<int> p(s.size(), 0);
	for(int i = 0; i < s.size(); i++) {
		if(maxr >= i) {
			p[i] = min(p[2 * mid - i], maxr - i);
		}
		while(s[i - p[i] - 1] == s[i + p[i] + 1])
		p[i]++;
		if(i + p[i] > maxr) {
			mid = i;
			maxr = i + p[i];
		}
		ans = max(ans, p[i]);
	}
	return ans;
}

int main() {
	string s;
	cin >> s;
	s = change(s);
	cout << manacher(s);
}
```


六、数论基础

6.1 费马小定理笔记

1. 定义与基本形式

定义：若 p 是质数，且 $\gcd(a,p)=1$，则：

$$a^{p-1} \equiv 1 \pmod{p}$$

一般形式：

$$a^{p} \equiv a \pmod{p}$$

2. 核心应用：模逆元

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

3. 快速幂模板

```cpp
#include <iostream>
using namespace std;
using i64 = long long;

i64 fast_pow(i64 base, i64 exponent, i64 mod) {
    i64 result = 1;
    base %= mod;
    while (exponent > 0) {
        if (exponent % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exponent /= 2;
    }
    return result;
}

i64 fermat_inverse(i64 a, i64 p) {
    return fast_pow(a, p - 2, p);
}

int main() {
    i64 a, p;
    cin >> a >> p;
    cout << fermat_inverse(a, p) << endl;
    return 0;
}
```


6.2 等比数列求和公式笔记

1. 基本公式

设等比数列首项为 a_1，公比为 q，项数为 n。

当 $q \neq 1$ 时：

$$S_n = \frac{a_1(1-q^n)}{1-q}$$

当 $q = 1$ 时：

$$S_n = na_1$$

2. 公式推导（错位相减法）

$$\begin{aligned} S_n &= a_1 + a_1q + a_1q^2 + \cdots + a_1q^{n-1} \\ qS_n &= \quad a_1q + a_1q^2 + \cdots + a_1q^{n-1} + a_1q^n \end{aligned}$$

两式相减：

$$S_n - qS_n = a_1 - a_1q^n$$

$$(1-q)S_n = a_1(1-q^n)$$

$$\therefore S_n = \frac{a_1(1-q^n)}{1-q}$$

3. 无穷等比数列（|q|<1）

$$S = \frac{a_1}{1-q}$$

4. 常用变形

已知条件	公式
已知 a_1、a_n	$S_n = \frac{a_1 - a_nq}{1-q}$
已知 a_1、a_n	$S_n = \frac{a_nq - a_1}{q-1}$

其中 $a_n = a_1q^{n-1}$。

6.3 GCD（最大公约数）

用法

求解两个整数的最大公约数，辗转相除递归即可。扩展欧几里得（exgcd）可求解不定方程 $ax + by = \gcd(a, b)$ 的一组特解。

完整模板

```
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}

int gcd(int a, int b) {
    if(b == 0) return a;
    else return gcd(b, a % b);
}

int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    int g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

差分数组

用法

差分数组主要用于解决区间批量修改的问题。它的核心思想是将对区间的操作转化为对两个单点（端点）的操作，从而显著降低时间复杂度。

实现步骤

对于差分数组有两个函数：

1. 初始化差分数组：$d_1 = a_1$，$d_i = a_i - a_{i-1}$（对于 $i > 1$）
2. 区间修改：执行 `d[L] += v; d[R + 1] -= v;`
3. 还原数组：对差分数组 d 求一遍前缀和，即可得到修改后的原数组 a

完整模板

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n = 5;
    vector<int> a = {0, 1, 2, 3, 4, 5}; // 假设下标从1开始
    vector<int> d(n + 2, 0);

    // 1. 构造差分数组
    for (int i = 1; i <= n; i++) {
        d[i] = a[i] - a[i - 1];
    }

    // 2. 区间修改：在区间 [2, 4] 上加 10
    int L = 2, R = 4, v = 10;
    d[L] += v;
    d[R + 1] -= v;

    // 3. 还原原数组（求前缀和）
    for (int i = 1; i <= n; i++) {
        a[i] = a[i - 1] + d[i];
        cout << a[i] << " ";
    }
    // 输出应为: 1 12 13 14 5

    return 0;
}
```

线段树

维护的信息类型

父范围的信息能从子范围得到 满足的如区间的 最大值 最小值 和 乘积等 不满足的如 范围上出现次数最多的数字

功能

范围查询 修改 时间复杂度logn

如何判断范围修改能否在logn时间内完成

就看在修改后能否在o(1)时间内得到修改后的信息

线段树的思想

首先 线段树是一个维护信息的完全二叉树
如图

线段树的构建过程是一个递归的过程
在这个过程中 存在函数f(l, r, i) 依次代表l, r 维护的区间的长度
i 存储这个区间信息的树的数组下标
递归f(l, m, 2 * i) f(m + 1, r, 2 * i + 1)就能维护线段树的左右区间并且以此放在左孩子和右孩子上



空间需要一般为4n 需要极限压缩可以为不大于 n 的最大 2 的幂次（即 $2^{\lfloor \log_2 n \rfloor}$）

代码结构

线段树的代码一般包含三个部分

1. `bulid` 函数 建立一开始的线段树
   用递归来完成
2. `query` 函数 查询线段树上的信息
   用递归来完成
   每次查询三种可能
   要查询的元素在不在当前区间上，在则认为命中了 直接返回
   如果部分在区间上 分为在左边 在右边的情况 依次判断 继续递归
3. `update` 函数 更新区间的信息 其中还包含up 和 down数组

完整模板

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// ============================================================
// 1. 业务逻辑区：打比赛时只需修改下面这三个部分
// ============================================================

struct Tag { //懒标记
    long long add = 0; // 例子：区间加法

    Tag(long long v = 0) : add(v) {}

    // 动作叠加：旧标记如何吸收新标记 t
    void apply(const Tag &t) {
        add += t.add;
    }
};

struct Info { //信息
    long long sum = 0;
    int len = 0; // 维护区间长度，方便计算区间加

    Info() {}
    Info(long long v, int l) : sum(v), len(l) {}

    // 动作作用于结果：标记 t 如何改变当前信息
    void apply(const Tag &t) {
        sum += (long long)len * t.add;
    }
};

// 合并：两个子区间如何合并成父区间
Info operator+(const Info &a, const Info &b) {
    return {a.sum + b.sum, a.len + b.len};
}

// ============================================================
// 2. 框架区：通用的带懒标记线段树（左闭右开）
// ============================================================

template<class Info, class Tag> //模版 传入信息和懒标记
struct LazySegmentTree {
    int n;
    vector<Info> info;
    vector<Tag> tag; //初始化变量和数组

    LazySegmentTree(int n) : n(n) {
        info.assign(4 << __lg(n), Info());
        tag.assign(4 << __lg(n), Tag());
    }//初始化懒标记数组和信息数组长度

    LazySegmentTree(vector<Info> init_v) : n(init_v.size() - 1) {
        info.assign(4 << __lg(n), Info());
        tag.assign(4 << __lg(n), Tag());

        auto build = [&](auto &self, int p, int l, int r) -> void {
            if (r - l == 1) {
                info[p] = init_v[l];
                return;
            }
            int mid = (l + r) >> 1;
            self(self, p << 1, l, mid);
```

```
            self(self, p << 1 | 1, mid, r);
            pull(p);
        };
        build(build, 1, 1, n + 1);
    } //构造函数 传入最初的数组

    void pull(int p) {
        info[p] = info[p << 1] + info[p << 1 | 1];
    } //向上传递信息

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);

    } //修改当前点的信息和懒标记信息

    void push(int p) {
        apply(p << 1, tag[p]);
        apply(p << 1 | 1, tag[p]);
        tag[p] = Tag();
    } // 向下传递信息

    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int mid = (l + r) >> 1;
        push(p);
        if (x < mid) modify(p << 1, l, mid, x, v);
        else modify(p << 1 | 1, mid, r, x, v);
        pull(p);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) return;
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int mid = (l + r) >> 1;
        push(p);
        rangeApply(p << 1, l, mid, x, y, v);
        rangeApply(p << 1 | 1, mid, r, x, y, v);
        pull(p);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) return Info();
        if (l >= x && r <= y) return info[p];
        int mid = (l + r) >> 1;
        push(p);
        // 性能优化：如果查询范围只在某一侧，不进行跨侧合并
        if (y <= mid) return rangeQuery(p << 1, l, mid, x, y);
        if (x >= mid) return rangeQuery(p << 1 | 1, mid, r, x, y);
        return rangeQuery(p << 1, l, mid, x, y) + rangeQuery(p << 1 | 1, mid, r, x, y);
    }

    // 外部调用接口（1-based）
    void modify(int p, const Info &v) { modify(1, 1, n + 1, p, v); }
    void rangeApply(int l, int r, const Tag &v) { rangeApply(1, 1, n + 1, l, r, v); }
    Info rangeQuery(int l, int r) { return rangeQuery(1, 1, n + 1, l, r); }
};

// ============================================================
// 使用示例
// ============================================================
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 5;
    // 准备初始数据，假设数组为 [1, 2, 3, 4, 5]
    vector<Info> a(n + 1);
    for (int i = 1; i <= n; i++) {
        a[i] = Info(i, 1); // 初始值 i，长度 1
    }

    LazySegmentTree<Info, Tag> st(a);

    // 区间 [1, 3] 加上 10 -> 注意左闭右开 rangeApply(1, 4, ...)
    st.rangeApply(1, 4, Tag(10));

    // 查询区间 [2, 4] 的和 -> rangeQuery(2, 5)
    cout << st.rangeQuery(2, 5).sum << endl;

    return 0;
}
```

0-1 BFS

适用范围

边权值为可能有，也可能没有（由于 BFS 适用于权值为 1 的图，所以一般权值是 0 或 1），或者能够转化为这种边权值的最短路问题．

例如在走迷宫问题中，你可以花 1 个金币走 5 步，也可以不花金币走 1 步，这就可以用 0-1 BFS 解决．

实现

一般情况下，我们把没有权值的边扩展到的点放到队首，有权值的边扩展到的点放到队尾．这样即可保证像普通 BFS 一样整个队列队首到队尾权值单调不下降．

离散化

用法

将值域很大的稀疏数据映射到连续的 $[1, m]$ 下标，保持原大小关系不变。常用于树状数组、线段树等需要以下标为键的场景。

实现步骤

1. 复制原数组，排序后去重，得到有序唯一值列表
2. 遍历原数组，用 `lower_bound` 查出每个值在列表中的排名（1-based），存入映射数组
3. 原值 ↔ 排名通过下标 `i` 对应： `a[i]` 是原值， `rnk[i]` 是排名

完整模板

```
// 1. 排序 + 去重
vector<int> v(a + 1, a + n + 1);
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
int m = v.size(); // 离散化后的值域大小

// 2. 映射原数组为排名（1-based），树状数组/线段树可用 rnk[i] 作下标
vector<int> rnk(n + 1);
for (int i = 1; i <= n; i++) {
    rnk[i] = lower_bound(v.begin(), v.end(), a[i]) - v.begin() + 1; //映射离散化后的排名 查询排名也用这种方法
}

// 3. 排名和原值的对应关系
//    原值 a[i]        -> 排名 rnk[i]
//    排名 k 对应的原值 -> v[k - 1]
```

图论基础

BFS（广度优先搜索）

用法

用于无权图最短路、拓扑排序、层序遍历，按层次逐层扩展，首次到达即为最短路。

完整模板

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> G;
vector<bool> vis;
vector<int> dis;

void bfs(int s) {
    queue<int> q;
    q.push(s);
    vis[s] = true;
    dis[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : G[u]) {
            if (!vis[v]) {
                vis[v] = true;
                dis[v] = dis[u] + 1;
                q.push(v);
            }
        }
    }
}
```

DFS（深度优先搜索）

用法

用于连通性判断、环检测、回溯、拓扑排序（后序），沿路径深入到底再回溯。

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> G;
vector<bool> vis;

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }
}
```

Dijkstra（单源最短路）

用法

求解非负权图单源最短路，时间复杂度 $\mathcal{O}(m \log n)$。

实现步骤

1. `dist[s] = 0`，其余为 INF
2. 小顶堆存 `(距离, 结点)`，起点入堆
3. 每次取出堆顶，跳过已确定的点，否则标记并松弛邻接点
4. 堆空即得起点到各点的最短路

完整模板

```
#include <bits/stdc++.h>
using namespace std;
int n, m, s;
vector<bool> vis;
int main() {
	cin >> n >> m >> s;
	vis.resize(n + 1, 0);
	vector<vector<pair<int, int>>> E(n + 1);
	for(int i = 1; i <= m; i++) {
		int u, v, w;
		cin >> u >> v >> w;
		E[u].push_back({w, v});
	}
	priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
	vector<int> dis(n + 1, 1e9 + 5);
	dis[s] = 0;
	pq.push({0,s});

	while (!pq.empty()) {
		auto [d, u] = pq.top();
		pq.pop();
		if(vis[u]) continue;
		vis[u] = 1;
		for(auto [w, v]:E[u]) {
			if(dis[v] > dis[u] + w) {
				dis[v] = dis[u] + w;
				pq.push({dis[v], v});
			}
		}
	}
	for(int i = 1; i <= n; i++) {
		cout << dis[i] << ' ';
	}

}
```

这份笔记涵盖了欧拉函数的定义、通式推导，以及如何将其嵌套进线性筛（欧拉筛）的完整逻辑。

📖 欧拉函数 $\varphi(n)$ 笔记

1. 定义与通式

定义：$\varphi(n)$ 表示 $[1, n]$ 中与 n 互质的整数个数。
通式：若 $n = p_1^{k_1} p_2^{k_2} \dots p_m^{k_m}$（质因数分解），则：

$$\varphi(n) = n \prod_{i=1}^{m} \left(1 - \frac{1}{p_i}\right)$$

2. 核心性质推导（用于线性筛）

线性筛的核心是通过 $n = i \times p$（p 为最小质因子）来递推。我们需要根据 i 是否包含 p 分两种情况：

情况 A：$p \mid i$ （p 整除 i）

- **前提**：i 已经包含了质因子 p。

- **推导**：
  根据通式，$\varphi(i)$ 已经包含了 $(1-1/p)$ 这个项。
  当计算 $\varphi(i \times p)$ 时，由于**质因子种类没有增加**，通式右边的乘积项 $\prod(1-1/p_i)$ 完全不变。
  唯一的改变是前面的系数从 i 变成了 $i \times p$。
- **结论**：

$$\varphi(i \times p) = \varphi(i) \times p$$

情况 B：$p \nmid i$ （p 不整除 i）

- **前提**：p 是一个全新的质因子，且 p 是质数，故 $\gcd(i, p) = 1$。
- **推导**：
  根据积性函数性质：$\varphi(i \times p) = \varphi(i) \times \varphi(p)$。
  由于 p 是质数，$\varphi(p) = p - 1$。
- **结论**：

$$\varphi(i \times p) = \varphi(i) \times (p - 1)$$

3. 线性筛欧拉函数标准模板 (C++)

```cpp
#include <iostream>
#include <vector>

using namespace std;

typedef long long i64;
const int MAXN = 20000005; // 根据题目要求修改

int phi[MAXN];
int primes[1500000]; // 预估素数数量
bool is_not_prime[MAXN];
int cnt = 0;

void precompute_phi(int n) {
    phi[1] = 1; // 1的欧拉函数定义为1
    for (int i = 2; i <= n; ++i) {
        if (!is_not_prime[i]) {
            primes[cnt++] = i;
            phi[i] = i - 1; // 性质：素数 p 的 phi(p) = p - 1
        }
        for (int j = 0; j < cnt; ++j) {
            i64 temp = (i64)i * primes[j];
            if (temp > n) break;

            is_not_prime[temp] = true;

            if (i % primes[j] == 0) {
                // 情况 A：primes[j] 是 i 的老面孔 (p | i)
                // 质因子种类没变，只增加数值倍数
                phi[temp] = phi[i] * primes[j];
                break; // 线性筛精髓：找到最小质因子后停止
            } else {
                // 情况 B：primes[j] 是新质因子 (p 不整除 i)
                // 利用积性函数性质 phi(i*p) = phi(i) * (p-1)
                phi[temp] = phi[i] * (primes[j] - 1);
            }
        }
    }
}

int main() {
    // 快速 I/O
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    precompute_phi(20000000);

    int T, x;
    if (!(cin >> T)) return 0;
    while (T--) {
        cin >> x;
        cout << phi[x] << "\n";
    }
    return 0;
}
```

4. 记忆要点总结

1. **初始化**： `phi[1] = 1` 。
2. **遇到素数**： `phi[i] = i - 1` 。
3. **内层标记**：
   - **能整除**：说明质因子已经在地盘里了，直接**乘** p。

- **不能整除**：说明来了个新质因子，要**乘** $(p-1)$。

这两个公式的推导，本质上是观察**“质因子的种类”**有没有发生变化。我们再次祭出欧拉函数的通式（也就是你图片里的那个）：

$$\varphi(n) = n \cdot \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

这个公式告诉我们：$\varphi(n)$ 的值由 n 本身 和 n 包含的所有不同质因子 共同决定。公式一：当 $i \pmod p = 0$ 时这种情况意味着 p 已经是 i 的质因子了。分析 $\varphi(i)$：

$$\varphi(i) = i \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdot \dots \cdot \left(1 - \frac{1}{\mathbf{p}}\right)$$

分析 $\varphi(i \cdot p)$：当我们把 i 乘以 p 得到新数时，新数的质因子集合与 i 的完全一样（因为 p 本来就在里面）。所以，公式右边的连乘部分（那些括号）一个都不会变。

$$\varphi(i \cdot p) = (i \cdot p) \cdot \underbrace{\left(1 - \frac{1}{p_1}\right) \cdot \dots \cdot \left(1 - \frac{1}{p}\right)}_{\text{这部分就是 } \frac{\varphi(i)}{i}}$$

得出结论：

$$\varphi(i \cdot p) = (i \cdot p) \cdot \frac{\varphi(i)}{i} = \varphi(\mathbf{i}) \cdot \mathbf{p}$$

直观理解：质因子种类没变，“打折比例”就没变，数值翻了 p 倍，结果自然也翻 p 倍。公式二：当 $i \pmod p \neq 0$ 时这种情况意味着 p 是一个全新的质因子，它不在 i 原有的质因子列表里。分析 $\varphi(i)$：

$$\varphi(i) = i \cdot \prod_{p_{old}|i} \left(1 - \frac{1}{p_{old}}\right)$$

分析 $\varphi(i \cdot p)$：因为 p 是新来的，根据通式，我们不仅要把前面的系数乘以 p，还要在后面多乘一个新括号 $(1 - \frac{1}{p})$。

$$\varphi(i \cdot p) = (i \cdot p) \cdot \left[\prod_{p_{old}|i} \left(1 - \frac{1}{p_{old}}\right)\right] \cdot \left(\mathbf{1} - \frac{\mathbf{1}}{\mathbf{p}}\right)$$

化简：中间那一团就是 $\varphi(i)$，所以：

$$\varphi(i \cdot p) = \varphi(i) \cdot p \cdot \left(1 - \frac{1}{p}\right)$$

把 p 乘进括号里：$p \times (1 - \frac{1}{p}) = p - 1$。

$$\varphi(\mathbf{i} \cdot \mathbf{p}) = \varphi(\mathbf{i}) \cdot (\mathbf{p} - \mathbf{1})$$

费马小定理

$$\text{每个数/2 向下取整的和} = \frac{\text{数组总和} - \text{奇数的个数}}{2}$$

七、筛法

7.1 欧拉筛（线性筛）

用法

欧拉筛（线性筛）用于在 $\mathcal{O}(n)$ 时间内求出 $[1, n]$ 范围内的所有素数，每个合数只被其最小质因子筛掉一次，避免了埃氏筛的重复标记。

实现步骤

1. 维护 `is_not_prime[]` 标记合数， `primes` 列表存储素数

2. 遍历 $i = 2 \dots n$：
   - 若 i 未被标记，则为素数，加入 `primes`
   - 遍历 `primes`，将 $i \times primes[j]$ 标记为合数
   - 若 $i \bmod primes[j] = 0$，则 **break**
3. 最终 `primes` 列表即为 $[1, n]$ 内所有素数

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 MAXN = 1e7 + 5; //需要的素数的范围 自行修改
vector<i64> primes; //求得的素数数组
bool is_not_prime[MAXN]; //不是素数的判断

void euler_sieve(int n) {
    is_not_prime[0] = is_not_prime[1] = true;//标记0和1
    for (int i = 2; i <= n; i++) {
        if (!is_not_prime[i]) {
            primes.push_back(i);
        }
        for (int p : primes) {
            if ((long long)i * p > n) break;
            is_not_prime[i * p] = true;
            if (i % p == 0) break; //合数 = 最小质数 * 一个数 保证当前是最小质数 才能保证合数的唯一性
        }
    }
}
```


八、欧拉路径与欧拉回路

8.1 判定条件

用法

判断一个图是否存在欧拉路径或欧拉回路（一笔画问题）：能否从一点出发，每条边恰好经过一次（欧拉回路要求最终回到起点）。

判定条件（图需连通/弱连通）

- 无向图：
  - 欧拉回路：所有点的度数均为偶数
  - 欧拉路径：恰有 0 或 2 个奇度点（恰有 2 个奇度点时，它们分别是起点和终点）
- 有向图：
  - 欧拉回路：所有点 入度 = 出度
  - 欧拉路径：恰有一个点 出度 - 入度 = 1（起点），恰有一个点 入度 - 出度 = 1（终点），其余点 入度 = 出度

8.2 Hierholzer算法

用法

在 $\mathcal{O}(n + m)$ 时间内求出一条欧拉路径/欧拉回路。

实现步骤

1. 选起点：欧拉回路任选一个有边的点；欧拉路径选出度 - 入度 = 1 的点（无向图选奇度点）
2. 从起点 DFS，每走过一条边就删掉这条边（无向图还要删掉反向边）

3. 走到无路可走时，将当前点压入答案
4. 答案逆序输出即为欧拉路径/回路

完整模板（有向图）

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

vector<vector<int>> G; // 邻接表
vector<int> indeg, outdeg;
vector<int> ans;
int n, m;

bool check() { // 有向图判定：是否存在欧拉路径/回路
    int start_cnt = 0, end_cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (outdeg[i] - indeg[i] == 1) start_cnt++;
        else if (indeg[i] - outdeg[i] == 1) end_cnt++;
        else if (indeg[i] != outdeg[i]) return false;
    }
    return (start_cnt == 0 && end_cnt == 0) || (start_cnt == 1 && end_cnt == 1);
}

void hierholzer(int u) {
    while (!G[u].empty()) {
        int v = G[u].back();
        G[u].pop_back();
        hierholzer(v);
    }
    ans.push_back(u); // 逆序输出 ans 即欧拉路径
}

int main() {
    cin >> n >> m;
    G.resize(n + 1);
    indeg.assign(n + 1, 0);
    outdeg.assign(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        outdeg[u]++;
        indeg[v]++;
    }
    if (!check()) {
        cout << "No" << endl;
        return 0;
    }
    int start = 1;
    for (int i = 1; i <= n; i++) {
        if (outdeg[i] - indeg[i] == 1) start = i;
    }
    hierholzer(start);
    reverse(ans.begin(), ans.end());
    for (auto &i : ans) {
        cout << i << ' ';
    }
    return 0;
}
```

完整模板（无向图）

```
vector<vector<pair<int, int>>> G; // (邻接点, 边编号)
vector<bool> used; // 边是否被使用过
vector<int> ans;

void dfs(int u) {
    while (!G[u].empty()) {
        auto [v, eid] = G[u].back();
        G[u].pop_back();
        if (used[eid]) continue;
        used[eid] = true;
        dfs(v);
    }
    ans.push_back(u);
}
```